

BÁO CÁO PHÂN TÍCH CHI TIẾT

.HVM0 / DNGuard HVM 4.9.6

Mục tiêu: tái dựng pipeline giải mã và khôi phục MethodBody hoàn toàn từ host

THÔNG TIN BÁO CÁO	
Mẫu phân tích	LordsMobileBot.exe / LordsMobileBot.dll
Protector	DNGuard HVM 4.9.6.0, x64
Phạm vi	Lookup, container, metadata, record, decrypt, final JIT IL, virtual-token resolver
Mốc báo cáo	05/08/2026
Trạng thái tổng quát	Host-only pipeline đã hoạt động cho hai known target; parser tổng quát còn nhiều family chưa giải

CONFIRMED Các kết luận xanh có bằng chứng runtime/offline khớp trực tiếp.

STRONG INFERENCE Các kết luận vàng có chuỗi bằng chứng mạnh nhưng chưa hoàn tất mapping tổng quát.

UNRESOLVED Các kết luận đỏ vẫn là khoảng trống kỹ thuật cần truy tiếp.

1. Tóm tắt điều hành

Nghiên cứu HVM0 đã chuyển từ giai đoạn phụ thuộc runtime dump sang một pipeline host-only có thể đọc trực tiếp LordsMobileBot.exe, phục hồi bảng tra method, giải nén hai container chính, dựng metadata runtime, lấy record của method, giải mã payload bằng XOR + RC4 và tạo raw IL suffix. Với method 0x06002CD4, TTD đã xác nhận thêm quá trình HVM phát sinh dynamic prefix, ghép final IL dài 0x2E byte và chuyển nó cho cljit. Phần virtual-token resolver cũng đã được lần tới custom HVM proxy và cache entry chứa CLR handles.

Kết quả quan trọng nhất không chỉ là “dump được IL”, mà là đã xác định được nhiều lớp định dạng và thuật toán đủ để triển khai offline: lookup formula, hai lớp decode của bảng, LZARI parameters, metadata post-transform, record layout, meaningful length, aligned size, RC4 key schedule, auxiliary block count và scheme của virtual token. Đây là nền tảng để tiến tới một unpacker/rebuilder có khả năng tái lập MethodBody mà không cần HVMSRun64.dll chạy trực tiếp.

Kết luận ở thời điểm báo cáo: Đã chứng minh end-to-end cho hai method known target và đã tách rõ ba bài toán còn lại: family-specific record layouts, dynamic prefix synthesis tổng quát và virtual-token → metadata token mapping.

1.1 Chỉ số tổng quan

Chỉ số	Giá trị	Ý nghĩa
MethodDef trong module	27.296	Toàn bộ RID được đưa qua scanner
Method trích xuất hoàn tất theo parser hiện tại	15.908	58,2796% toàn module; 84,49% trong prefix đến boundary
Method thất bại	11.388	Chủ yếu tail unmapped và family/layout chưa giải
Known target được runtime xác nhận	2	0x060015E2 và 0x06002CD4
Final JIT IL đã bắt trực tiếp	0x2E byte	Method 0x06002CD4
Virtual slots quan sát	1..5	FieldDef, TypeRef, MemberRef-like operands

1.2 Mức độ sẵn sàng

Hạng mục	Trạng thái	Nhận xét
Host-only container extraction	CONFIRMED	Hoạt động trên mẫu hiện tại; hash runtime/offline khớp.
Method record lookup	CONFIRMED	Công thức RID → directory RVA → recordOffset đã ổn định.
XOR + RC4 payload decode	CONFIRMED	Runtime trace khớp byte-for-byte với offline output.
Final JIT IL reconstruction	CONFIRMED cho 0x06002CD4	Prefix 0x14 + suffix 0x1A = 0x2E.
Batch extraction generalization	PARTIAL	58,28% toàn module; nhiều family chưa có parser đúng.
Virtual-token offline patching	UNRESOLVED	Đã có handles runtime, chưa có metadata token thật.

2. Phạm vi, mục tiêu và tiêu chuẩn bằng chứng

2.1 Mục tiêu nghiên cứu

- Loại bỏ phụ thuộc vào dump HVMSRun64.dll và các blob runtime để trích xuất method.
- Đọc toàn bộ dữ liệu cần thiết trực tiếp từ LordsMobileBot.exe.
- Tái dựng exact MethodBody mà cljit nhìn thấy, không chỉ raw suffix sau decrypt.

- Phân biệt token CLR thật và virtual token do HVM tạo.
- Xây dựng nền tảng cho rebuilder có khả năng patch IL vào managed module tĩnh.

2.2 Tiêu chuẩn phân loại kết luận

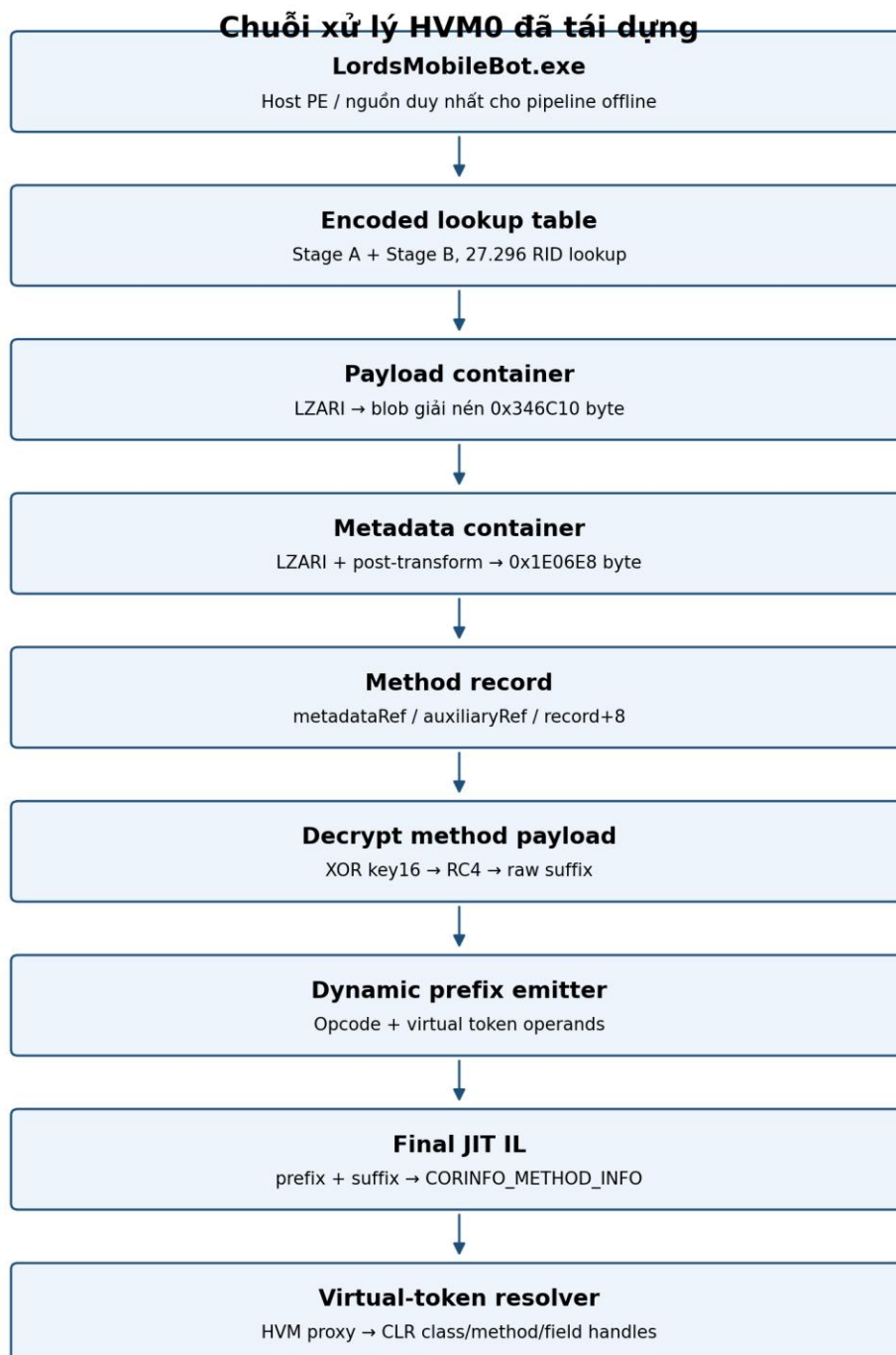
Nhãn	Tiêu chuẩn	Ví dụ trong báo cáo
CONFIRMED	Có ít nhất một runtime trace hoặc hash/byte comparison trực tiếp.	RC4 output 0x20 byte; final JIT IL 0x2E byte.
STRONG INFERENCE	Nhiều bằng chứng nhất quán, chưa có trace đầy đủ cho mọi nhánh.	Vai trò low 28 bit của auxiliary item type 5.
UNRESOLVED	Chưa đủ dữ liệu để đặt semantics hoặc xây thuật toán tổng quát.	Mapping virtual slot → metadata token thật.

2.3 Mẫu và môi trường

MÔI TRƯỜNG ĐÃ XÁC NHẬN	
Host	D:\LordsBot-Release\LordsMobileBot.exe
Host SHA-256	2178BC077A362A18DBDC5B141478740F0870FEDC2774E2ED0D741C606F318A0E
Managed payload	LordsMobileBot.dll
Runtime DLL	C:\Users\LENOVO\AppData\Local\Temp\DN235159646A24E428\HVMRun64.dll
HVM version	4.9.6.0 x64
HVM image base / size	0x180000000 / 0x3A4000
Runtime	.NET 8.0.29 / CoreCLR + clrjit
Debugger	WinDbg TTD

3. Kiến trúc HVM0 đã tái dựng

Chuỗi xử lý hiện đã được hiểu ở mức đủ để triển khai host-only extraction cho sample. Các khối màu xanh trong sơ đồ là phần đã có thuật toán hoặc runtime evidence trực tiếp; tầng resolver cuối vẫn cần truy mapping metadata token thật.



Hình 1. Chuỗi xử lý HVM0 đã tái dựng từ host đến clrjit.

3.1 Bảng lookup method

Bảng lookup đã được phục hồi trực tiếp từ host bằng hai lớp biến đổi. Vùng decoded có kích thước 0x1B878 byte và cung cấp một DWORD cho mỗi MethodDef RID.

LOOKUP TABLE	
Host stage0 offset	0xB1349C
Decoded region size	0x1B878
Stage0 SHA-256	B46F06E392201EA9820CA4425FCBBB24619A41378A8C7CA2B56D1EECFB25236E
Stage1 SHA-256	4EC0DDB39D1640EB30C8D80407FC7D4EEB57872D57482B1C2D57FE8C61D34683
Decoded SHA-256	57C43D9E191F8A3F21879706B2DEFFE0592B6D68AB527FBD1700D70192FCFB9A

```
// Stage A
for (i = n - 1; i >= 1; --i)
    buffer[i] ^= buffer[i - 1];
buffer[0] ^= buffer[n - 1];

// Stage B
static const uint8_t key16[16] = {
    0x5F, 0xAA, 0x95, 0x01, 0xA4, 0x61, 0xBC, 0x81,
    0x05, 0x8E, 0x63, 0x52, 0x2B, 0xC6, 0x69, 0x7A
};
decoded[0] = stage1[0] ^ key16[0];
for (i = 1; i < n; ++i)
    decoded[i] = stage1[i] ^ key16[i & 0xF] ^ stage1[i - 1];
```

```
RID          = token & 0x0FFFFFFF;
encodedValue = read_u32(decodedRegion + (RID - 1) * 4);
directoryRva = (encodedValue ^ 0x23511004) & 0x07FFFFFF;
```

3.2 Directory entry và record

```
struct HvmDirectoryEntry {
    uint8_t tag;
    uint8_t flags;
    uint32_t recordOffset;
}; // 6 bytes

struct HvmRecord {
    uint32_t metadataRef; // high 24 bit = metadata offset; low 8 bit = flags
    uint32_t auxiliaryRef; // sentinel hoặc offset/flag tùy mode
    uint8_t adjacentData[];
};
```

TTD đã xác nhận `record+8` là vùng dữ liệu được theo dõi riêng, đồng thời low byte của metadataRef là bitfield. Với metadataRef 0x0485AC03, runtime thực hiện `shr r15d,8` để lấy 0x485AC và kiểm tra nhiều bit riêng lẻ của AL. Vì vậy không nên gọi low byte là “type tag” đơn giản.

3.3 Packed metadata và meaningful length

```
packedMetadata = read_u32(metadataBase + metadataOffset);
metadataType   = packedMetadata & 0xFF;
meaningfulLength = packedMetadata >> 8;
alignedSize     = (meaningfulLength + 7) & ~7;
```

Đối với 0x06002CD4, runtime đọc 0x00001A02 tại metadata+0x485AC rồi chạy `shr ecx,8`, xác nhận trực tiếp meaningfulLength = 0x1A. Aligned size là 0x20 và cả 0x20 byte được XOR, copy và RC4; chỉ 0x1A byte đầu được giữ làm raw IL suffix.

4. Hai container và thuật toán LZARI

4.1 Payload container

PAYLOAD BLOB	
Host offset	0xB2ED14
Output size	0x346C10
Compressed stream	0xB2ED18, size 0x322673
Container end	0xE5138B
Zero-padding bits	9
Output SHA-256	1E007A5B90F5BF8AFA2EA86C248A877CDA6156233859FE35E5D9D4646E0ED3E7

4.2 Metadata container

METADATA BLOB	
Host offset	0xE5138B
Compressed size	0x1CFA27
Output size	0x1E06E8
Intermediate SHA-256	24E75FF3DC7B11451AE238DFA4667ED2C1A2EE889BA3B71513FB6877B41C0A60
Final SHA-256	809333CB66FB64622E7AF9F5F1D32836CBCA3B19BA66F7C0C41D34CAA0A62284
Runtime metadata base	0x24104631040

```
// Metadata post-transform
output[0] = input[0] ^ key16[0];
for (i = 1; i < size; ++i)
    output[i] = input[i] ^ key16[i & 0xF] ^ input[i - 1];
```

4.3 LZARI parameters đã xác định

```
N = 4096
F = 60
THRESHOLD = 2
N_CHAR = 314
M = 15
Q1 = 1 << 15
Q2 = 2 * Q1
Q3 = 3 * Q1
Q4 = 4 * Q1

symbol = decode_char();
if (symbol < 0x100) {
    emit_literal(symbol);
```

```

} else {
    distance = decode_position();
    src = (writeIndex - distance - 1) & 0xFFF;
    length = symbol - 253;
    copy_from_ring(src, length);
}

```

5. Chuỗi giải mã method payload

5.1 XOR stage

```

for (i = 0; i < alignedSize; ++i)
    xorBuffer[i] = recordAdjacentData[i] ^ key16[i & 0xF];

```

TTD tại method 0x06002CD4 bắt source byte đầu 0x15, key byte 0x5F và output 0x4A. Toàn bộ 0x20 byte output trùng tuyệt đối với offline implementation.

5.2 RC4 variant

Key RC4 dài 20 byte: 16-byte key cố định nối với seed little-endian `04 10 51 23`. KSA có thêm hằng số +1; PRGA là chuẩn và chạy in-place trên buffer aligned.

```

key20 = key16 || { 0x04, 0x10, 0x51, 0x23 };

// KSA
j = 0;
for (i = 0; i < 256; ++i) {
    j = (j + S[i] + key20[i % 20] + 1) & 0xFF;
    swap(S[i], S[j]);
}

// PRGA
for each byte {
    i++;
    j += S[i];
    swap(S[i], S[j]);
    k = S[(uint8_t)(S[i] + S[j])];
    data[pos] ^= k;
}

```

Runtime routine tại HVMRUN64+0x164C0 được xác nhận là PRGA. Byte đầu 0x4A XOR keystream 0x61 = 0x2B; state lưu S[256] cùng i và j tại offset +0x100/+0x101.

5.3 Padding behavior

Sáu byte cuối của aligned block không phải zero trước RC4. Chúng có ciphertext `8F D5 5A 19 EE 98`, sau RC4 trở thành sáu byte zero. Điều này xác nhận decrypt phải chạy trên alignedSize, sau đó cắt theo meaningfulLength.

6. Hai case study đã xác nhận end-to-end

6.1 Method 0x060015E2 – LordsMobileBot.appUtil.get_AppDirectory

KNOWN TARGET 1	
RID	0x15E2
Encoded value	0x03506BB2
Directory RVA / host offset	0x17BB6 / 0x6B7DB6
Directory tag / flags	FE / 0F
Record offset	0x1F01F8
Record header	0350AA0320800D00
Metadata flags / offset	03 / 0x3AA50
Packed metadata	0x00000E02
Meaningful / aligned size	0x0E / 0x10
Payload offset	0x1F0200
Raw IL	0028040080060A38000000000062A
Raw IL SHA-256	A73FBD0AE8063E2FC1DEF32DCEB2632103029EFCE2B4C14C5CE22B51A CB55B0A

Kết quả: Host-only extractor trả đúng expected IL và known-target PASS.

6.2 Method 0x06002CD4

KNOWN TARGET 2	
RID	0x2CD4
Encoded value	0x0350F2B0
Directory RVA / host offset	0x1E2B4 / 0x6BE4B4
Directory tag / flags	FE / 0F
Record offset	0x2DE950
Record header	03AC850458DC1500
metadataRef	0x0485AC03
auxiliaryRef	0x0015DC58
Packed metadata	0x00001A02
Meaningful / aligned size	0x1A / 0x20
Raw suffix	2B05289D70154100280400800A280500800A0A38000000000062A
Raw suffix SHA-256	38A5B1AEFF65CDE6B9DDCF97C1A3FEED28051CEE8E8BF5D7D2ECBD8 D52FF11D2

6.3 Final JIT IL của 0x06002CD4

TTD tại `clrjit!CILJit::compileMethod` cho `ILCodeSize = 0x2E`. Buffer cuối gồm prefix động 0x14 byte cộng raw suffix 0x1A byte.

```
Prefix 0x14:
00 7F 01 00 80 04 FE 16 02 00 80 01 6F 03 00 80 0A 2D 01 00

Suffix 0x1A:
2B 05 28 9D 70 15 41 00 28 04 00 80 0A 28 05 00
80 0A 0A 38 00 00 00 00 06 2A

Final 0x2E:
00 7F 01 00 80 04 FE 16 02 00 80 01 6F 03 00 80
0A 2D 01 00 2B 05 28 9D 70 15 41 00 28 04 00 80
0A 28 05 00 80 0A 0A 38 00 00 00 00 06 2A
```

FINAL METHODBODY EVIDENCE	
Final IL length	0x2E (46 byte)
Final IL SHA-256	9C7F87FBFEFAE2FA2AA774DDD3CED1574055CD5D5C934E91E3146D5F57CA3730A
Prefix source workspace	0x24100669774
Final buffer	0x2410066A290
Suffix destination	final+0x14 = 0x2410066A2A4

6.4 Dynamic prefix emitter

Prefix không xuất hiện nguyên văn trong metadata blob, payload blob hay HVM image. TTD cho thấy các opcode và operand được ghi ở nhiều vị trí thời gian khác nhau vào một workspace tái sử dụng, sau đó block-copy vào final IL. Ba operand token được emitter ghi trực tiếp từ register.

Offset	Opcode/operand	Giá trị	Bằng chứng
+0x00	nop	00	Scalar writer trong prefix workspace
+0x01	ldsflda	7F	Writer riêng
+0x02	FieldDef-like virtual token	0x04800001	EDX ghi DWORD
+0x06	constrained.	FE16	Opcode writer riêng
+0x08	TypeRef-like virtual token	0x01800002	ESI ghi DWORD
+0x0C	callvirt	6F	Opcode writer riêng
+0x0D	MemberRef-like virtual token	0x0A800003	EAX ghi DWORD

7. Auxiliary block và resolver-side metadata

7.1 AuxiliaryRef lookup

Với 0x06002CD4, auxiliaryRef không phải offset trực tiếp vào payload blob. Runtime dùng nó làm byte offset trong một auxiliary table nằm bên trong metadata.

```
mode = 1
auxTableOffset = 0x5A8E0
auxKey = 0x6A24E41B
```

```

auxiliaryRef    = 0x0015DC58

entryOffset = auxTableOffset + auxiliaryRef; // byte offset, không nhân 4
entryValue  = read_u32(metadata + entryOffset); // 0x6A24E419
decodedAux   = entryValue ^ auxKey;           // 2
auxFlag      = auxiliaryRef & 1;              // 0

```

AUXILIARY BLOCK	
Lookup metadata offset	0x1B8538
Table value	0x6A24E419
Decoded auxiliary count	2
Aux flag	0
Entries pointer runtime	0x241047E957C

7.2 Encoded auxiliary entries

Helper nhận `entries = auxEntry+4` và `count = 2`, sau đó đọc đúng hai DWORD. Mỗi item được XOR với `index+2`, lấy high nibble để dispatch.

```

for (uint32_t index = 0; index < count; ++index) {
    uint32_t iterationMask = index + 2;
    uint32_t decodedItem = encodedItems[index] ^ iterationMask;
    uint32_t itemType = decodedItem >> 28;
    uint32_t payload = decodedItem & 0xFFFFFFFF;
}

```

Index	Encoded	Mask	Decoded	Type	Payload
0	0x5A696ACD	2	0x5A696ACF	5	0x0A696ACF
1	0x5A686893	3	0x5A686890	5	0x0A686890

Giới hạn hiện tại: Đã xác nhận count, loop và type=5; semantics của low 28 bit vẫn chưa được giải.

8. Virtual token scheme và runtime resolver

8.1 Định dạng virtual token

Virtual token giữ nguyên byte table category của CLR metadata và bật marker 0x00800000. Phần thấp là slot trong resolver của HVM.

```
isVirtual = (token & 0x00800000) != 0;
table     = token & 0xFF000000;
slot      = token & 0x007FFFFF;
```

Token	Table category	Marker	Slot	Vị trí
0x04800001	FieldDef-like (0x04)	0x00800000	1	Dynamic prefix
0x01800002	TypeRef-like (0x01)	0x00800000	2	Dynamic prefix
0x0A800003	MemberRef-like (0x0A)	0x00800000	3	Dynamic prefix
0x0A800004	MemberRef-like (0x0A)	0x00800000	4	Raw suffix
0x0A800005	MemberRef-like (0x0A)	0x00800000	5	Raw suffix

Cảnh báo: Không được chỉ xóa bit 0x00800000 rồi dùng RID thấp làm metadata RID thật. Slot của HVM chưa được chứng minh trùng RID CLR.

8.2 HVM resolveToken proxy

clrjit đọc token tại `Compiler::impResolveToken` và gọi qua vtable slot +0xE0. Target là HVMRUN64+0x35100. Proxy kiểm tra marker 0x00800000: token bình thường được chuyển cho CoreCLR resolver gốc, token ảo được gửi tới custom resolver HVMRUN64+0x7850.

```
tokenTable = token & 0xFF000000;
if ((token & 0x00800000) != 0) {
    ResolveVirtualToken(hvmContext, token, callerAddress); // HVMRUN64+0x7850
} else {
    originalICorJitInfo->resolveToken(request);
}
```

8.3 Kết quả resolve slot 1

CORINFO_RESOLVED_TOKEN	
Virtual token	0x04800001
Token kind	4
Resolved class handle	0x7FFA7500C700
Resolved method handle	0
Resolved field handle	0x7FFA74FC4198
TypeDef token của class	0x02000F3E
Module	LordsMobileBot.dll / 0x7FFA74E9E0A0

HVM không rewrite trường token. Nó tạo triple handle `{hClass, hMethod, hField}` rồi copy vào request+0x18. Vì đây là ldsfda, hMethod bằng zero và hField có giá trị.

8.4 Cache entry của virtual token

```
struct VirtualTokenCacheEntry {
    uint8_t unknown[0x28];
    void *hClass;    // +0x28
    void *hMethod;   // +0x30
    void *hField;    // +0x38
};

entry = 0x24100668B90;
entry->hClass = 0x7FFA7500C700;
entry->hMethod = 0;
entry->hField = 0x7FFA74FC4198;
```

Cache entry được điền từ một buffer tạm chứa ba handles. Writer của cache field đã được tìm tới HVMRUN64+0x59F8/+0x59FC, nhưng nơi CoreCLR/HVM tạo ra triple handle ban đầu vẫn là checkpoint tiếp theo.

9. Kết quả batch scan và family analysis

9.1 Thống kê toàn module

SCANNER SUMMARY	
MethodDef scanned	27.296
Extracted	15.908
Failed	11.388
Extraction rate toàn module	58,2796%
Inferred boundary RID	0x498D
Rows tại/trước boundary	18.829
Extracted tại/trước boundary	15.908
Prefix extraction rate	84,49%
Rows sau boundary	8.467; tất cả unmapped

9.2 Phân loại failure

Nhóm	Số lượng	Đánh giá
Meaningful size = 0	1.905	Empty/alternate descriptor; không mặc định là lỗi decode.
Unmapped RVA trước boundary	777	Descriptor/layout chưa giải hoặc directory mismatch.
Encrypted payload OOB	238	Current record layout/length interpretation không phù hợp.
Record header OOB	1	Outlier đơn lẻ.
Tail sau boundary unmapped	8.467	Strong candidate ngoài current HVM directory mapping; chưa gọi tuyệt đối non-HVM.

9.3 Family distribution

Family / category	Số lượng	Nhận xét
ref03 plausible other meta	9.271	Đa số prefix methods; semantics metadata type chưa giải.

Family / category	Số lượng	Nhận xét
alternate_ref08	4.948	Current length interpretation thường tạo size cực lớn; layout khác.
alternate_ref00	661	Một số mẫu cho IL hợp lý; cần family parser riêng.
ref03 meta02	1.026	Tên classifier cũ “confirmed” quá rộng; chỉ 2 exact method đã TTD-confirmed.
prefix empty/alt descriptor	1.905	Không nên tự động coi là method rỗng.
tail non-HVM candidate	8.467	Đều unmapped sau RID 0x498D.

Diễn giải đúng: “Extracted” trong scanner chỉ có nghĩa parser/decrypt chain hoàn tất; không đồng nghĩa final CLR IL hợp lệ hoặc token đã remap.

10. Các cột mốc kỹ thuật quan trọng

Mốc	Nội dung	Bảng chứng đạt được	Ý nghĩa
M1	Xác định runtime và version	HVMRun64.dll 4.9.6.0 x64; image base/size; host hash.	Thiết lập sample identity và khả năng tái lập.
M2	Phục hồi lookup table từ host	Tìm stage0 offset, Stage A/B và formula RID→directory RVA.	Loại phụ thuộc vào lookup dump runtime.
M3	Tái dựng LZARI payload container	Output 0x346C10 và exact SHA-256.	Có payload blob hoàn toàn offline.
M4	Tái dựng metadata container	LZARI + post-transform; exact runtime hash.	Loại phụ thuộc metadata dump.
M5	Xác định record/metadata length model	metadataRef high24/low8 flags; packed metadata type/length.	Biết chính xác số byte cần decrypt.
M6	Xác định XOR + RC4	Key, KSA +1, PRGA, aligned padding.	Raw method suffix khớp runtime byte-for-byte.
M7	Host-only extractor known target	0x060015E2 và 0x06002CD4 known-target PASS.	Chứng minh end-to-end offline chain.
M8	Batch scan toàn module	27.296 RID; 15.908 extracted; boundary 0x498D.	Định lượng coverage và family gaps.
M9	Auxiliary table decode	auxTableOffset, auxKey, count=2, item loop index+2.	Bắt đầu hiểu resolver-side metadata.
M10	Bắt final JIT IL	Prefix 0x14 + suffix 0x1A = final 0x2E.	Phân biệt raw suffix với MethodBody mà clrjit nhận.
M11	Dynamic prefix emitter	Opcode/token được emit riêng vào workspace.	Mở đường tái tạo prefix offline.
M12	Virtual-token resolver	Marker 0x00800000; custom resolver; CLR handle cache.	Xác định chính xác khoảng trống cuối của rebuilder.

11. Pipeline host-only hiện tại

11.1 Thành phần đã có

Artifact	Vai trò	Trạng thái
recover_encoded_table_source.py	Khôi phục source lookup stage từ host.	Đã dùng
recover_encoded_table_two_layers.py	Decode Stage A + Stage B.	Đã dùng
decode_hvm_metadata.py	LZARI metadata + post-transform.	Known hash PASS
extract_hvm_host_only.py	Host-only method extractor.	2 known target PASS
scan_hvm_methods.py	Scan toàn bộ MethodDef RID.	Hoàn tất 27.296 rows
analyze_hvm_scan.py / deep.py	Thống kê boundary/family/failure.	Hoàn tất
classify_hvm_methods.py	Phân loại sơ bộ.	Cần đổi tên category "confirmed"

11.2 Pseudocode end-to-end hiện tại

```

host = read_file("LordsMobileBot.exe")

lookup = decode_lookup(host[0xB1349C : 0xB1349C + 0x1B878])
payload = lzari_decode_payload(host)
metadata_intermediate = lzari_decode_metadata(host)
metadata = metadata_post_transform(metadata_intermediate, key16)

for token in target_tokens:
    rid = token & 0xFFFFFFFF
    directory_rva = (lookup[rid-1] ^ 0x23511004) & 0x07FFFFFF

```

```

dir_entry = read_directory_entry(host, directory_rva)
record = payload + dir_entry.recordOffset

metadata_offset = read_u32(record) >> 8
metadata_flags = read_u32(record) & 0xFF
packed = read_u32(metadata + metadata_offset)
meaningful = packed >> 8
aligned = align8(meaningful)

xor_block = xor_key16(record + 8, aligned)
rc4_block = hvm_rc4(xor_block, key20)
raw_suffix = rc4_block[:meaningful]

# Family-specific work remains:
prefix = synthesize_prefix(record, auxiliary, resolver_metadata)
final_il = prefix + raw_suffix
final_il = remap_virtual_tokens(final_il)

```

12. Những phần chưa hoàn tất và rủi ro kỹ thuật

Khoảng trống	Trạng thái	Tác động
Family ref08/ref00 và metadata types khác	UNRESOLVED	Không thể gọi 15.908 output là MethodBody hoàn chỉnh.
Dynamic prefix synthesis tổng quát	PARTIAL	Đã chứng minh một method; chưa có grammar/template cho mọi subtype.
Virtual slot → metadata token thật	UNRESOLVED	CLR handles runtime không thể patch trực tiếp vào DLL.
Auxiliary item type 5 low 28-bit payload	UNRESOLVED	Chưa biết descriptor/token mapping được mã hóa ra sao.
Locals signature và EH	PARTIAL/UNRESOLVED	Cần bắt final CORINFO_METHOD_INFO / resolver calls theo family.
Boundary 0x498D	STRONG INFERENCE	Tail unmapped rất nhất quán nhưng chưa chứng minh tuyệt đối non-HVM.

12.1 Nguyên tắc không được overclaim

- Không gọi toàn bộ 1.026 ref03/meta02 là runtime-confirmed; hiện chỉ có hai exact method được TTD xác nhận.
- Không coi flags != 0x0F là decrypt đúng chỉ vì output trông giống IL.
- Không coi token bỏ marker 0x00800000 là metadata token thật.
- Không coi tail sau 0x498D tuyệt đối non-HVM; chỉ là non-HVM candidate theo current mapping.
- Không coi raw suffix của 0x06002CD4 là final MethodBody; final IL có thêm dynamic prefix 0x14 byte.

13. Kế hoạch tiếp tục

1. Truy writer nguồn của triple handle tại buffer 0xFD8257AF58, bắt nơi hField 0x7FFA74FC4198 được tạo.
2. Lần ngược từ hField/FieldDesc đến metadata token thật hoặc resolver descriptor chứa token/module.
3. Lập trace cho virtual slots 2, 3, 4, 5 để xác định type/method/member mapping và quy luật table/slot.
4. Giải low 28 bit của auxiliary item type 5, đối chiếu với slot/token operands trong final IL.
5. Tái tạo dynamic prefix offline cho 0x06002CD4 và thêm known-target final-IL hash.
6. Mở rộng parser theo family ref08/ref00 và metadata type khác; thêm IL syntax validation tách biệt với decrypt success.
7. Sau khi có token mapping, patch final MethodBody vào DLL rebuild và xác nhận dnSpy/CLR load không cần HVM runtime.

13.1 Checkpoint để tiếp tục TTD

```
# Mốc đã xác nhận cache field/class writer
!tt AD7CF:FC

# Buffer triple handle nguồn
r @$t17 = 0x000000fd8257af58
dq @$t17 L3

# Truy writer của hField nguồn
# (chạy từ vị trí sau khi triple đã được điền)
dx @$curprocess.TTD.PrevMemoryAccess("w",@$t17+0x10,8)
dx @$curprocess.TTD.PrevMemoryAccess("w",@$t17+0x10,8).Position.SeekTo()
t-
u @rip L30
r
k
```

14. Kết luận

Nghiên cứu HVM0 đã đạt bước chuyển quan trọng từ “capture/dump runtime” sang “tái dựng định dạng và thuật toán”. Lookup table, hai container, metadata transform, method payload crypto và final JIT IL của method đại diện đã được xác nhận bằng cả offline hash và TTD runtime evidence. Virtual token không còn là black box hoàn toàn: marker, slot, proxy, cache entry và CLR handles đã được xác định.

Khoảng trống cốt lõi còn lại là chuyển các virtual slot và resolver descriptors thành metadata token thật, đồng thời tổng quát hóa dynamic prefix/family layouts. Khi hai phần này được giải, pipeline hiện tại có đủ nền tảng để trở thành một unpacker/rebuilder host-only hoàn chỉnh cho sample DNGuard 4.9.6 đang nghiên cứu.

Đánh giá cuối: Kiến trúc và thuật toán chính đã được kiểm soát; bài toán còn lại tập trung, đo được và có checkpoint TTD rõ ràng. Đây là trạng thái nghiên cứu tiến triển cao nhưng chưa nên tuyên bố “full unpack mọi HVM method”.

Phụ lục A. Bản đồ bằng chứng truy vết

Nhóm bằng chứng	TTD / dữ liệu tiêu biểu	Điểm được chứng minh
RC4 input/output	11746E:BE0 → 11746E:DF7	PRGA in-place, aligned 0x20, output exact.
Final JIT IL	117470:A3F	IL pointer 0x2410066A290, size 0x2E.
Suffix copy	11746F:17C9–17CC	Source RC4 output → final+0x14 bitwise.
Prefix copy	11746F:2374–2383	Workspace 0x24100669774 → final buffer.
Aux count/items	11746F:2399; 117470:142/7C8	Count=2, đúng hai entries, mask=index+2.
Virtual token read	117471:67D	clrjit impResolveToken đọc 0x04800001.
Custom resolver	117471:6BE → 117479:24F	Marker branch vào HVMRUN64+0x7850.
Handle cache copy	117479:EB–F0	Cache entry +0x28/+0x30/+0x38 → triple handle.
Cache entry writer	AD7CF:FC–101	Triple nguồn → cache entry 0x24100668B90.

Phụ lục B. Nguồn log và artifact tham chiếu

Báo cáo được tổng hợp từ các log WinDbg TTD, kết quả scanner/classifier và các script host-only đã tạo trong chuỗi nghiên cứu. Các file log gần nhất tương ứng Pasted text (68) đến Pasted text (76); các kết quả nền trước đó gồm classification-summary.json và các TTD logs xác nhận record, metadata, XOR/copy và auxiliary lookup.

Nguồn	Nội dung chính
classification-summary.json	Boundary, category counts và scanner classification.
Pasted text (68)	Suffix/prefix writers và decodedAux read.
Pasted text (69)	RC4 source → suffix; prefix workspace; auxiliary helper call.
Pasted text (70)	Aux entries count=2; helper không patch final IL.
Pasted text (71)	Aux item decode và virtual token writers.
Pasted text (72)	iterationMask=index+2 và clrjit resolveToken entry.
Pasted text (73)	HVM marker branch; custom resolver identification.
Pasted text (74)	CORINFO_RESOLVED_TOKEN handles và cache copy.
Pasted text (75)	Cache entry layout +0x28/+0x30/+0x38.
Pasted text (76)	Cache entry writer, SOS class/module information.